

```

import os
import logging
import unicodedata
from telegram import Update
from telegram.ext import ApplicationBuilder, CommandHandler, MessageHandler, filt

logging.basicConfig(format="%(asctime)s - %(levelname)s - %(message)s", level=log

TOKEN = "8955757103:AAHShN1V8jD7eJbUh5lFolp0X7uKSLW8fwo"

ADMINS_RAW = os.environ.get("ADMIN_IDS", "")
ADMIN_IDS = [int(x.strip()) for x in ADMINS_RAW.split(",") if x.strip()]

UNIDADES = {}
for bloco in ["A", "B"]:
    for andar in range(1, 4):
        for apto in range(1, 4):
            nome = f"Apto {andar}0{apto}"
            chave = f"{nome}-{bloco}"
            UNIDADES[chave] = {"unidade": nome, "bloco": f"Bloco {bloco}", "tipo"

def is_admin(user_id):
    return not ADMIN_IDS or user_id in ADMIN_IDS

def sem_acento(s):
    return ''.join(c for c in unicodedata.normalize('NFD', s) if unicodedata.cate

def buscar_unidade(nome):
    nome = nome.strip().upper()
    for chave, u in UNIDADES.items():
        if u["unidade"].upper() == nome:
            return chave, u
    return None, None

async def cmd_start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    msg = ("Bot - Antonio Borges 122\n\n"
           "/disponiveis - Unidades disponiveis\n"
           "/todas - Todas as unidades\n"
           "/resumo - Resumo geral\n\n"
           "Admins:\n"
           "/reservar Apto 101 Nome\n"
           "/vender Apto 101 Nome\n"
           "/liberar Apto 101")
    await update.message.reply_text(msg)

```

```

async def cmd_disponiveis(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    disp = [u for u in UNIDADES.values() if u["status"] == "DISPONIVEL"]
    if not disp:
        await update.message.reply_text("Nenhuma unidade disponivel no momento.")
        return
    linhas = [f"OK {u['unidade']} - {u['bloco']} ({u['tipo']})" for u in disp]
    msg = "Unidades Disponiveis - Antonio Borges 122\n\n" + "\n".join(linhas)
    msg += f"\n\nTotal: {len(disp)} disponivel(is)"
    await update.message.reply_text(msg)

async def cmd_todas(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    icones = {"DISPONIVEL": "OK", "RESERVADO": "RESERVADO", "VENDIDO": "VENDIDO"}
    linhas = []
    bloco_atual = ""
    for u in UNIDADES.values():
        if u["bloco"] != bloco_atual:
            bloco_atual = u["bloco"]
            linhas.append(f"\n-- {bloco_atual} --")
        ic = icones.get(u["status"], "?")
        comp = f" -> {u['comprador']}" if u["comprador"] else ""
        linhas.append(f"{ic} {u['unidade']}{comp}")
    msg = "Todas as Unidades - Antonio Borges 122\n" + "\n".join(linhas)
    await update.message.reply_text(msg)

async def cmd_resumo(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    total = len(UNIDADES)
    disp = sum(1 for u in UNIDADES.values() if u["status"] == "DISPONIVEL")
    res = sum(1 for u in UNIDADES.values() if u["status"] == "RESERVADO")
    vend = sum(1 for u in UNIDADES.values() if u["status"] == "VENDIDO")
    pct = ((res + vend) / total * 100) if total else 0
    msg = (f"Resumo - Antonio Borges 122\n\n"
           f"Total: {total}\n"
           f"Disponiveis: {disp}\n"
           f"Reservadas: {res}\n"
           f"Vendidas: {vend}\n"
           f"Ocupado: {pct:.0f}%")
    await update.message.reply_text(msg)

async def cmd_reservar(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        await update.message.reply_text("Apenas admins podem fazer isso.")
        return
    args = ctx.args
    if len(args) < 3:
        await update.message.reply_text("Uso: /reservar Apto 101 Nome Completo")
        return

```

```

nome = f"{args[0]} {args[1]}"
comprador = " ".join(args[2:])
chave, u = buscar_unidade(nome)
if not u:
    await update.message.reply_text(f"Unidade {nome} nao encontrada.")
    return
UNIDADES[chave]["status"] = "RESERVADO"
UNIDADES[chave]["comprador"] = comprador
await update.message.reply_text(f"{nome} reservada para {comprador}.")

async def cmd_vender(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        await update.message.reply_text("Apenas admins podem fazer isso.")
        return
    args = ctx.args
    if len(args) < 3:
        await update.message.reply_text("Uso: /vender Apto 101 Nome Completo")
        return
    nome = f"{args[0]} {args[1]}"
    comprador = " ".join(args[2:])
    chave, u = buscar_unidade(nome)
    if not u:
        await update.message.reply_text(f"Unidade {nome} nao encontrada.")
        return
    UNIDADES[chave]["status"] = "VENDIDO"
    UNIDADES[chave]["comprador"] = comprador
    await update.message.reply_text(f"{nome} vendida para {comprador}.")

async def cmd_liberar(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        await update.message.reply_text("Apenas admins podem fazer isso.")
        return
    args = ctx.args
    if len(args) < 2:
        await update.message.reply_text("Uso: /liberar Apto 101")
        return
    nome = f"{args[0]} {args[1]}"
    chave, u = buscar_unidade(nome)
    if not u:
        await update.message.reply_text(f"Unidade {nome} nao encontrada.")
        return
    UNIDADES[chave]["status"] = "DISPONIVEL"
    UNIDADES[chave]["comprador"] = ""
    await update.message.reply_text(f"{nome} esta DISPONIVEL novamente.")

async def resposta_livre(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    texto = sem_acento((update.message.text or "").lower())

```

```

palavras_disp = ["disponivel", "disponiveis", "livre", "livres", "quais", "qu
palavras_resumo = ["resumo", "quantas", "total", "quanto", "vendida", "vendid
if any(p in texto for p in palavras_disp):
    await cmd_disponiveis(update, ctx)
elif any(p in texto for p in palavras_resumo):
    await cmd_resumo(update, ctx)
else:
    await update.message.reply_text("Use /disponiveis, /todas ou /resumo.")

def main():
    app = ApplicationBuilder().token(TOKEN).build()
    app.add_handler(CommandHandler("start", cmd_start))
    app.add_handler(CommandHandler("disponiveis", cmd_disponiveis))
    app.add_handler(CommandHandler("todas", cmd_todas))
    app.add_handler(CommandHandler("resumo", cmd_resumo))
    app.add_handler(CommandHandler("reservar", cmd_reservar))
    app.add_handler(CommandHandler("vender", cmd_vender))
    app.add_handler(CommandHandler("liberar", cmd_liberar))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, resposta_livr
    print("Bot rodando v3")
    app.run_polling()

if __name__ == "__main__":
    main()

```